

PLANO DE TESTES AUTOMATIZADOS

ServeRest API (serverest.dev) - Testes Funcionais de API Automatizados com Postman

Item	Detalhe
Autor	Luiz Vinicius Cunha Maciel
Projeto	Portfólio pessoal de QA
Versão	2.0 - Automatizada (baseada no Plano de Testes Manual v1.0)
Data	01/08/2026
Ferramenta de automação	Postman (Collection Runner)

1. Introdução

Este Plano de Testes Automatizados refatora o Plano de Testes Manual v1.0, elaborado para a API pública ServeRest (<https://serverest.dev>), convertendo os 32 casos de teste manuais (CT-01 a CT-32) em uma switch de testes automatizados executável via Postman.

A ServeRest expõe recursos de usuários, produtos e carrinhos de compra, com regras de negócio (exclusividade de e-mail, controle de estoque, permissões de administrador, vínculo entre carrinho e usuário/produto). A automação elimina a repetição manual dessas verificações, tornando a switch executável a qualquer momento, de forma consistente e sem dependência de estado deixado por execuções anteriores.

2. Especificações do Projeto

2.1. Objetivo Geral

Automatizar a validação das principais funcionalidades da ServeRest - login, cadastro, consulta, atualização e exclusão de usuários e produtos, e gestão de carrinho de compras - garantindo, a cada execução da suíte, que os endpoints retornem os status codes, mensagens e estruturas de dados esperados, com feedback rápido e repetível.

2.2. Aplicação Sob Teste

Item	Detalhe
Nome da aplicação	ServeRest — https://serverest.dev
Tipo	API REST simulando back-end de e-commerce
Ambiente	Produção (ambiente público de demonstração)
Formato de dados	JSON (request e response)
Autenticação	Token bearer obtido via POST /login, enviado no header Authorization
Variável de ambiente	URL = https://serverest.dev ; token (preenchido dinamicamente em runtime)

2.3. Funcionalidades em Escopo

Mesmo escopo funcional do plano manual - login, usuários, produtos e carrinhos - agora coberto por 32 requisições automatizadas organizadas em 5 pastas da collection Postman:

Pasta (Folder)	Endpoint(s)	Casos cobertos	Foco
Login	POST /login	CT-01, CT-02, CT-03, CT-04	Autenticação, geração e validação do bearer token
Cadastro de Usuários	/usuarios (GET, GET/{id}, POST, PUT, DELETE)	CT-05, CT-06, CT-07, CT-08, CT-09, CT-10, CT-11, CT-12, CT-13, CT-14, CT-15	CRUD de usuários, unicidade de e-mail, filtros, vínculo com carrinho
Produtos	/produtos (GET, GET/{id}, POST, PUT, DELETE)	CT-16, CT-17, CT-18, CT-19, CT-20, CT-21, CT-22	CRUD de produtos, autenticação e permissão de administrador
Carrinhos	/carrinhos (GET, GET/{id}, POST, DELETE concluir/cancelar)	CT-23, CT-24, CT-25, CT-26, CT-27, CT-28, CT-32	Criação, consulta, conclusão e cancelamento de compra, controle de estoque
Segurança	Rotas protegidas diversas	CT-29, CT-30, CT-31	Acesso sem token, token inválido e exposição de dados sensíveis

2.4. Fora do Escopo

- Testes de interface do front-end (front.serverest.dev)
- Testes de carga e performance em larga escala
- Testes de segurança avançados
- Infraestrutura, deploy e monitoramento da aplicação
- Massa de dados fixa/hardcoded - toda a massa é gerada dinamicamente em runtime (ver seção 3.2)

3. Estratégia de Automação

3.1. Abordagem

A switch foi construída na ferramenta Postman, com os 32 casos de teste manuais convertidos em requisições automatizadas, organizadas nas mesmas 5 pastas funcionais do plano original. Cada requisição é autocontida: os dados necessários (usuários, produtos, carrinhos) são criados dinamicamente em scripts de pré-requisição (pre-request scripts), eliminando a dependência de massa de dados fixa ou de estado deixado por execuções anteriores.

Essa abordagem permite executar a collection inteira, uma pasta isolada, ou um único teste, em qualquer ordem, com resultado determinístico - requisito importante dado que o ambiente público da ServeRest reinicia sua base de dados diariamente.

3.2. Geração Dinâmica de Massa de Dados

Um conjunto de funções utilitárias (helpers), replicado no pre-request script de cada requisição, monta os pré-requisitos de cada caso de teste em tempo de execução:

- randomEmail() / randomNome() / randomSenha() - geram dados únicos por execução (timestamp + valores aleatórios do Postman, ex.: {{\$randomFullName}})
- criarUsuario(administrador) - cria via POST /usuarios um usuário comum ou administrador

- login(email, senha) - autentica via POST /login e retorna o token
- criarUsuarioAdminComToken() - combina os dois helpers acima para obter um admin autenticado
- criarProduto(token, overrides) - cadastra um produto via POST /produtos com dados padrão sobrescritíveis
- criarCarrinho(token, idProduto, quantidade) - cria um carrinho via POST /carrinhos
- criarUsuarioComCarrinho(idProduto, quantidade) - monta o cenário completo para os testes de vínculo e regra de negócio (ex.: CT-15, CT-22, CT-24)

Exemplo de helper (trecho do pre-request script, comum a toda a collection):

```
async function criarUsuario(administrador) {
  const nome = randomNome();
  const email = randomEmail();
  const senha = randomSenha();
  const res = await sendRequestAsync({
    url: BASE_URL + "/usuarios", method: "POST",
    header: { "Content-Type": "application/json" },
    body: { mode: "raw", raw: JSON.stringify({ nome, email, password: senha,
      administrador: administrador ? "true" : "false" }) }
  });
  return { nome, email, senha, id: res.json()._id };
}
```

Isso resolve, por construção, a restrição de suspensão do plano manual relacionada à reinicialização diária da base, cada execução recria a própria massa.

3.3. Estrutura das Verificações (Assertions)

Cada requisição contém um script de teste (test script), executado com a biblioteca Chai embutida no Postman, cobrindo consistentemente quatro camadas de verificação:

- Status code - validação do código HTTP esperado (200, 201, 400, 401, 403)
- Tempo de resposta - limite de performance funcional (< 2000ms) como guarda-corpo básico
- Contrato/schema - presença, tipo e a não-ausência dos campos obrigatórios no body de request/response
- Regra de negócio - mensagens de erro/sucesso específicas (ex.: "Este email já está sendo usado", "Não é permitido ter mais de 1 carrinho")

Exemplo de test script (CT-01 - Login com credenciais válidas):

```
pm.test("Status code e 200", () => pm.response.to.have.status(200));
pm.test("Tempo de resposta aceitável (< 2000ms)", () =>
  pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Resposta contém authorization com Bearer token", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("authorization");
  pm.expect(json.authorization).to.match(/^Bearer\s.+\/);
});
```

3.4. Técnicas de Design Mantidas da Automação

- Particionamento de equivalência e análise de valor limite - preservados nos dados gerados (ex.: e-mails únicos vs. duplicados, quantidade solicitada > estoque)
- Tabela de decisão para regras de negócio - automatizada via variação do parâmetro administrador (true/false) nos helpers de criação de usuário

- Casos de uso ponta-a-ponta - fluxo cadastro → login → produto → carrinho → conclusão de compra, orquestrado pelos helpers em cadeia (criarUsuarioComCarrinho)
- Teste de contrato/schema - assertions dedicadas à estrutura do JSON de resposta, mantidas de forma explícita em cada requisição

4. Cobertura de Casos de Teste

Os 32 casos de teste do plano manual foram mapeados 1:1 para requisições automatizadas (100% de cobertura), preservando o ID (CT-XX) como nome de cada requisição na collection, o que garante rastreabilidade direta entre o documento de casos de teste manual e a switch automatizada.

Prioridade (plano manual)	Casos de teste	Status na automação
Alta (Crítica)	CT-01, CT-02, CT-03, CT-05, CT-06, CT-15 a CT-18, CT-22 a CT-25, CT-27 a CT-31	Automatizados - executados a cada run
Média	CT-04, CT-07 a CT-10, CT-13, CT-14, CT-19, CT-21, CT-26, CT-32	Automatizados - executados a cada run
Baixa	CT-11, CT-12, CT-20	Automatizados - executados a cada run

5. Ambiente de Testes e Ferramentas

Categoria	Ferramenta/Recurso
Execução de testes	Postman (Collection Runner) - desenvolvimento e execução manual
Documentação da API	Swagger interativo — https://serverest.dev
Arquivos da switch	Testes_postman_collection_automatizada.json (collection) + ServeRest_environment.json (environment: URL, token)
Geração de dados de teste	Scripts de pre-request em JavaScript + variáveis dinâmicas do Postman ({{\$randomFullName}} etc.)

6. Critérios de Entrada e Saída

6.1. Critérios de Entrada

- Ambiente <https://serverest.dev> acessível
- Collection e environment do Postman importados e validados

6.2. Critérios de Saída

- 100% dos 32 casos de teste automatizados executados com sucesso
- Nenhuma falha (failure) ou erro (error) reportado no resumo do Postman

6.3. Critério de Suspensão e Retomada

A execução é suspensa caso o ambiente <https://serverest.dev> fique indisponível ou caso o rate limit da API seja atingido. Como toda a massa de dados é gerada em runtime, não há dependência de estado de execuções

anteriores, a switch pode ser retomada a qualquer momento assim que o ambiente for restabelecido, sem necessidade de reconfiguração.

7. Papéis e Responsabilidades

Papel	Responsabilidade
Analista de Teste / SDET	Manter a collection e os os helpers de massa de dados

8. Entregáveis

- Plano de Testes Automatizados (este documento)
- Collection Postman automatizada (Testes_postman_collection_automatizada.json)
- Arquivo de environment (ServeRest_environment.json)